

Prix au m² dans l'Hérault — rapport de projet

Binôme : jasondahmoun · r.pintre

IPSSI Montpellier — Mastère Dév, Data & IA, 4^e année — module MIA4

Date : 28 août 2026

Dépôt : https://github.com/jasondahmoun/projet_final_nosql_jason_romain

i) Problématique

Contexte métier

Une agence immobilière de l'agglomération de Montpellier veut objectiver ses estimations. Ses négociateurs raisonnent aujourd'hui sur des prix « au ressenti », commune par commune, sans référence chiffrée opposable au vendeur. La question qu'ils posent est simple : *combien vaut réellement le m² ici, et comment ça bouge ?*

La donnée existe et elle est publique : l'État publie chaque semestre les Demandes de Valeurs Foncières (DVF), c'est-à-dire l'intégralité des mutations immobilières enregistrées par les notaires. Le besoin est donc un besoin d'exploitation, pas de collecte.

Les trois questions métier

Ces trois questions ont été formulées avant l'écriture du moindre pipeline.

1. Quelles sont les 10 communes de l'Hérault où le m² s'est vendu le plus cher en 2024, en ne retenant que les communes ayant enregistré au moins 30 ventes ?

Le seuil de 30 n'est pas décoratif : sans lui, une commune où deux villas se sont vendues remonte en tête du classement et le rend inutilisable.

2. Comment le prix moyen au m² a-t-il évolué mois par mois en 2024, séparément pour les maisons et pour les appartements ?

La distinction maison / appartement est le premier réflexe du métier : ce sont deux marchés qui ne bougent pas ensemble.

3. Quels appartements se sont vendus dans un rayon de 5 km autour du centre de Montpellier, et à quel prix au m² ?

C'est la question du négociateur devant un bien précis : *qu'est-ce qui s'est vendu autour, récemment, et à combien ?*

Le jeu de données

Source	Demandes de Valeurs Foncières, <code>data.gouv.fr</code> (fichier géocodé <code>geo-dvf</code>)
URL	<code>https://files.data.gouv.fr/geo-dvf/latest/csv/2024/departements/34.csv.gz</code>
Périmètre	Département 34 (Hérault), année 2024
Volume brut	69 651 lignes (69 652 avec l'en-tête)
Après regroupement	29 565 mutations distinctes
Licence	Licence Ouverte / Open Licence (Etalab)

Le fichier convient aux trois questions parce qu'il porte, sur chaque ligne, les trois dimensions dont elles ont besoin : un prix (`valeur_fonciere`), une surface (`surface_reelle_bati`) et une position (`longitude`, `latitude`). Rien n'a à être enrichi depuis une autre source.

Pourquoi MongoDB pour ce cas précis

L'argument « le module portait sur MongoDB » n'en est pas un. Trois traits de cette donnée justifient réellement le modèle document :

Une mutation est un agrégat naturel, pas une ligne. Le CSV éclate une vente sur autant de lignes qu'elle comporte de lots ou de parcelles — jusqu'à 329 lignes pour une seule mutation dans notre jeu. En relationnel, on modéliserait `mutation` et `lot` en deux tables jointes à chaque lecture. En document, ce qui est lu ensemble est stocké ensemble : un document `mutations` porte sa vente et ses lots. Toutes nos requêtes lisent la mutation entière ; aucune ne lit un lot seul.

Le schéma est irrégulier par nature. 10 186 mutations sur 29 565 n'ont aucun type de local (terrains nus, dépendances seules), 44 850 lignes ont une surface bâtie vide, 565 n'ont pas de coordonnées. Un schéma relationnel strict imposerait une colonne nullable pour chacun de ces cas ; le modèle document laisse simplement le champ absent, et le validateur `$jsonSchema` reste posé là où il a du sens.

La question 3 est géospatiale. Un index `2dsphere` et un `$geoNear` répondent nativement à « dans un rayon de 5 km, trié par distance ». En PostgreSQL il faudrait PostGIS ; ici c'est un type de champ et une ligne d'index.

ii) Problèmes de données rencontrés

Aucun jeu réel n'est propre. Nous avons passé sur les données les cinq sondes de détection systématiques, et chaque anomalie trouvée est chiffrée ci-dessous.

Anomalie	Comment détectée	Ampleur	Traitement retenu	Justification
Une mutation étalée sur plusieurs lignes	<code>countDocuments()</code> vs <code>distinct("id_mutation").length</code>	69 651 lignes pour 29 565 mutations · 17 698 mutations (60 %) sur plus d'une ligne · maximum : 329 lignes pour <code>2024-406722</code>	<code>\$group</code> par <code>id_mutation</code> avant tout calcul, <code>valeur_fonciere</code> prise une seule fois (<code>\$first</code>)	Sans regroupement, une mutation étalée sur 8 lignes pèse 8 fois dans la moyenne. Mesuré sur Montpellier : facteur 3,60 (voir ci-dessous)
Types mixtes : champs numériques stockés en chaîne	<code>countDocuments({ champ: { \$type: "string" } })</code> sur <code>raw</code>	<code>surface_reelle_bati</code> : 44 850 · <code>code_postal</code> : 1 189 · <code>longitude</code> : 565 · <code>valeur_fonciere</code> : 420	<code>\$convert ... onError: null</code> à la transformation	<code>mongoimport --headerline</code> infère les types ligne par ligne : une cellule vide devient une chaîne vide, et <code>"" > 9</code> est faux sans erreur. La comparaison échoue en silence, ce qui est pire qu'un plantage
Locaux hétérogènes dans une même mutation	<code>\$addToSet</code> sur <code>type_local</code> puis <code>\$setIntersection</code>	99 mutations mêlent Maison et Appartement · 10 186 n'ont aucun local habitable	Surface sommée uniquement sur les lignes Maison/Appartement ; <code>type_local = "Mixte"</code> quand les deux coexistent, exclu de Q1 et Q2	Un <code>\$first</code> sur <code>type_local</code> typait la mutation selon l'ordre des lignes. Une vente Appartement + Dépendance devenait « Dépendance » une fois sur deux, et sa surface était additionnée au mauvais type

Anomalie	Comment détectée	Ampleur	Traitement retenu	Justification
Valeurs aberrantes de prix au m ²	<code>\$min / \$max sur prix_m2</code>	Étendue de 0,001 à 104 208 €/m² sur le périmètre habitable · 58 mutations hors de <code>[100 ; 20 000]</code>	Bornes <code>100 < prix_m2 < 20 000</code>	Un studio de 10 m ² déclaré à 1 M€ n'est pas une erreur de saisie repérable ligne à ligne, mais il déplace une moyenne communale. La médiane y résiste, la moyenne de Q2 non
Mutations qui ne sont pas des ventes	<code>\$group sur nature_mutation</code>	178 échanges · 89 adjudications · 182 ventes de terrain à bâtir, soit 449 mutations (111 dans le périmètre habitable)	Filtre <code>nature_mutation ∈ {Vente, VEFA}</code>	La <code>valeur_fonciere</code> d'un échange est une valeur d'expertise, pas un prix de marché ; celle d'une adjudication est un prix de vente forcée. Les mélanger aux ventes biaise le prix vers le bas
Intégrité référentielle	<code>\$lookup vers communes puis \$match: { c: { \$size: 0 } }</code>	0 mutation orpheline	Aucun	MongoDB n'impose aucune clé étrangère : l'absence d'orphelin devait être vérifiée, pas supposée. Elle s'explique ici par le fait que <code>communes</code> est dérivée de <code>mutations</code>

Le chiffre naïf et le chiffre correct

La requête qu'on écrit spontanément — moyenne du rapport prix / surface, directement sur les lignes brutes, sans filtre — donne, sur Montpellier :

```
naïf      (raw, sans filtre, sans regroupement) : 12 045 €/m2 sur 13 272 lignes
correct (mutations, filtré, regroupé)          : 3 346 €/m2 sur 4 157 mutations
écart : +260 % (facteur 3,60)
```

Le pipeline naïf est faux pour trois raisons cumulées, du plus lourd au plus léger :

- 1. Il divise un prix de mutation par la surface d'un seul lot.** Sur une vente à 300 000 € portant trois lots de 60 m², chacune des trois lignes calcule $300\,000 / 60 = 5\,000$ €/m² au lieu de $300\,000 / 180 = 1\,667$ €/m².
- 2. Il pondère par le nombre de lignes.** Une mutation sur 329 lignes pèse 329 fois plus qu'une mutation sur une ligne.
- 3. Il n'écarte ni les dépendances ni les caves,** dont la surface bâtie déclarée est vide ou minuscule.

Les deux valeurs sont publiées ici volontairement : ce n'est pas la bonne qui est instructive, c'est l'écart.

Ce que ces anomalies changent pour l'interprétation

Après traitement, **19 072 mutations sur 29 565** entrent dans l'analyse de prix — soit 65 % du jeu. Les 35 % écartés ne sont pas du bruit : ce sont des terrains, des dépendances seules et des biens sans surface bâtie déclarée. Nos résultats ne décrivent donc **que le marché du logement bâti**, et ne disent rien du foncier nu, qui représente une part significative des transactions de l'Hérault.

Deuxième réserve : DVF ne couvre pas l'Alsace-Moselle ni Mayotte, et exclut les mutations à titre gratuit (donations, successions). Un « prix moyen » calculé ici est un prix de vente entre vifs, pas une valeur de patrimoine.

iii) Démarche

Modélisation

Deux collections.

`mutations` — 29 565 documents. L'`_id` est l'`id_mutation` : l'unicité est garantie par construction, sans index supplémentaire ni contrainte à maintenir.

```
{ _id: "2024-403665",
  date_mutation: ISODate("2024-08-14"),
  valeur_fonciere: 104500,
  nature_mutation: "Vente",
  nom_commune: "Sète",
  code_commune: 34301,
  code_postal: 34200,
  type_local: "Appartement",
  surface_reelle_bati: 29,
  nb_lignes: 2,
  nb_locaux: 1,
  prix_m2: 3603.45,
  geo: { type: "Point", coordinates: [3.699835, 43.411157] } }
```

`communes` — 340 documents (l'Hérault en compte 342 ; deux communes n'ont enregistré aucune mutation en 2024).

```
{ _id: 34172, nom: "Montpellier", code_postal: 34000,
  nb_mutations: 4218,
  centroide: { type: "Point", coordinates: [3.8767, 43.6108] } }
```

Relation	Décision	Cardinalité	Taille bornée	Cycle de vie
mutation → ses lignes / lots	imbriqué	1 à 329, médiane 2	oui, largement sous 16 Mo	identique — un lot n'existe pas sans sa vente
mutation → commune	référéncé	1 à 4 218	non	indépendant — le référentiel évolue sans les mutations

Ce qui nous ferait changer d'avis. Sur l'imbrication : une mutation portant plusieurs milliers de lots — un programme neuf entier vendu en bloc — approcherait la limite BSON de 16 Mo. Notre maximum observé est 329 lignes, soit trois ordres de grandeur de marge ; la décision tiendrait même en multipliant le volume par 10. Sur la référence : si l'application ne lisait jamais que le *nom* de la commune, dénormaliser ce seul champ dans `mutations` supprimerait le `$lookup` pour un coût de duplication négligeable. C'est d'ailleurs déjà le cas — `nom_commune` est présent dans les deux collections — et le `$lookup` ne sert qu'aux requêtes qui ont besoin du reste du référentiel.

Import

```
curl -L -o dvf34.csv.gz https://files.data.gouv.fr/geo-dvf/latest/csv/2024/departements/34.csv.gz
gunzip -kf dvf34.csv.gz
mongoimport --uri "$URI" --collection raw --type csv --headerline --drop --file dvf34.csv
mongosh "$URI" --file db/02-transform.js
```

Transformations appliquées, dans l'ordre : conversion des types (`$convert` avec `onError: null`), regroupement par `id_mutation`, somme des surfaces sur les seules lignes habitables, détermination du `type_local` par intersection d'ensembles, calcul de `prix_m2`, construction du champ `geo` en GeoJSON [`longitude`, `latitude`], puis dérivation de `communes` par `$group` + `$out`.

L'ordre [`longitude`, `latitude`] de GeoJSON est l'inverse de l'ordre usuel « latitude, longitude ». Inversé, l'index se construit sans erreur et la carte pointe au Kazakhstan.

Indexation

Requête mesurée : `db.mutations.find({ code_postal: "34000" }).` Captures complètes dans `rapport/captures/explain_avant.json` et `explain_apres.json`.

	Avant (aucun index)	Après {code_postal:1, date_mutation:-1}
chaîne de stages	COLLSCAN	FETCH ← IXSCAN
totalKeysExamined	0	1 933
totalDocsExamined	29 565	1 933
nReturned	1 933	1 933
ratio examinés / rendus	15,3	1,0
executionTimeMillis	16	4

L'index ne divise pas seulement le temps par quatre : il fait passer le ratio examinés / rendus de 15,3 à 1,0. MongoDB ne lit plus que les documents qu'il retourne. C'est ce chiffre qui compte, pas la milliseconde — le temps dépend du cache et de la machine, le ratio non.

Index	Requête servie	Coût en écriture
{ code_postal: 1, date_mutation: -1 }	filtrage par secteur, tri par date (Q1, Q2)	une entrée B-tree par écriture sur <code>mutations</code>
{ geo: "2dsphere" }	<code>\$geoNear</code> de Q3 — obligatoire , sans lui la requête échoue au lieu d'être lente	le plus cher des trois : structure géospatiale recalculée à chaque écriture touchant <code>geo</code>
{ nom_commune: 1 }	GET /mutations?commune=... du CRUD	une entrée B-tree par écriture

Les trois index sont créés **après** le `$out` du transform, une fois les 29 565 documents en place : leur coût d'écriture ne pèse pas sur l'import en masse, seulement sur les écritures unitaires du CRUD.

Trois index, pas dix. Chacun sert une requête nommée ; un index créé « au cas où » coûte en écriture et ne se justifie devant personne.

Attention à la lecture du plan : le stage **racine** d'une requête indexée est `FETCH`, et l' `IXSCAN` est son `inputStage`. Rapporter le seul stage racine ferait écrire « `COLLSCAN` → `FETCH` », ce qui ne prouve rien.

Architecture applicative

Front statique	→ API REST	→ Driver	→ MongoDB
HTML/JS	→ FastAPI	→ PyMongo	→ 2 collections + 3 index
tableau·graphe	→ CRUD + /agg/*	→ pool unique	→ utilisateur applicatif
carte Leaflet	validation	BSON↔JSON	auth activée

Un seul `MongoClient` pour tout le processus : il gère lui-même son pool de connexions. En instancier un par requête est l'erreur classique et se paie en latence de handshake.

Sécurité

- **Utilisateur applicatif** `readWrite` sur la seule base du projet, créé au premier démarrage par `db/01-init-app-user.js`. Jamais `root`, aucun droit sur `admin` ni sur les autres bases.
- **Secrets** : `MONGO_URI` construite depuis l'environnement, `.env` ignoré par git, `.env.example` seul versionné. Vérification : `git log -p | grep -i "mongodb://"` ne renvoie aucun identifiant réel.
- **Validation des entrées** par Pydantic — aucun dictionnaire brut n'est inséré. Codes HTTP 400, 404, 409 et 422 gérés explicitement.
- **CORS** restreint à l'origine exacte du front, jamais `*`.

iv) Résultats

Question 1 — Les 10 communes les plus chères

Quelles sont les 10 communes de l'Hérault où le m² s'est vendu le plus cher en 2024, parmi celles ayant enregistré au moins 30 ventes ?

```
db.mutations.aggregate([
  // on ne garde que les ventes de logements bâtis, hors valeurs aberrantes
  { $match: {
    nature_mutation: { $in: ["Vente", "Vente en l'état futur d'achèvement"] },
    type_local: { $in: ["Maison", "Appartement"] },
    surface_reelle_bati: { $gt: 9 },
    valeur_fonciere: { $gt: 1000 },
    prix_m2: { $gt: 100, $lt: 20000 } } },

  // médiane et non moyenne : elle résiste aux quelques ventes hors marché
  { $group: {
    _id: "$code_commune",
    commune: { $first: "$nom_commune" },
    median_m2: { $median: { input: { $divide: ["$valeur_fonciere", "$surface_reelle_bati"] },
                          method: "approximate" } },
    n: { $sum: 1 } } },

  // seuil de significativité : sous 30 ventes, une villa suffit à fausser le classement
  { $match: { n: { $gte: 30 } } },
  { $sort: { median_m2: -1 } },
  { $limit: 10 },

  // jointure vers le référentiel, APRÈS le $limit : elle ne porte que sur les
  // 10 documents retournés, pas sur les 340 communes
  { $lookup: { from: "communes", localField: "_id", foreignField: "_id", as: "ref" } },
  { $unwind: { path: "$ref", preserveNullAndEmptyArrays: true } },

  { $project: { _id: 0, commune: 1, n: 1,
    median_m2: { $round: ["$median_m2", 0] },
    centroide: "$ref.centroide" } }
])
```

C'est le `$lookup` significatif exigé au § 1.2. Sa **position dans le pipeline est le point à défendre** : placé après le `$sort` et le `$limit`, il ne réalise que 10 jointures. Placé avant le `$group`, il en aurait réalisé une par mutation — 19 072 — pour un résultat identique. Le `preserveNullAndEmptyArrays` garde la ligne même si le référentiel est incomplet : un classement amputé d'une commune serait un bug silencieux, un `centroide` absent est visible.

#	Commune	Prix médian €/m²	Ventes retenues
1	La Grande-Motte	5 400	664
2	Palavas-les-Flots	5 135	234
3	Vic-la-Gardiole	5 128	65
4	Lattes	4 581	224
5	Mauguio	4 542	325

Interprétation. Le classement est intégralement littoral ou périelittoral : les cinq premières communes sont à moins de 8 km de la mer. Le marché héraultais se structure par la distance à la côte avant de se structurer par la taille de la commune — Montpellier, de loin la plus grosse en volume, n'apparaît pas dans les dix premières, sa médiane étant tirée vers le bas par un parc ancien étendu.

Réserves. Le seuil de 30 ventes exclut mécaniquement les petites communes de l'arrière-pays, dont certaines pourraient être chères ; le classement décrit donc le marché **actif**, pas le marché cher. La médiane masque la dispersion : Vic-la-Gardiole tient sa 3^e place sur 65 ventes seulement, contre 664 pour La Grande-Motte, et son intervalle de confiance est bien plus large.

Question 2 — Évolution mensuelle

Comment le prix moyen au m² a-t-il évolué mois par mois en 2024, séparément pour les maisons et pour les appartements ?

```
{ $group: {
  _id: { mois: { $dateToString: { format: "%Y-%m", date: "$date_mutation" } },
    type: "$type_local" },
  prix_m2_moyen: { $avg: { $divide: ["$valeur_fonciere", "$surface_reelle_bati"] } },
  n: { $sum: 1 } } }
```

Affiché en courbe sur le front, une série par type de local.

Mois	Appartement	Maison	Écart	Ventes
2024-01	3 341	2 913	+14,7 %	1 295
2024-03	3 582	2 879	+24,4 %	1 310
2024-05	3 419	2 784	+22,8 %	1 451
2024-09	3 481	3 005	+15,8 %	1 739
2024-12	3 515	2 976	+18,1 %	2 041

Interprétation. L'appartement se vend systématiquement plus cher au m² que la maison, sur les douze mois sans exception : de +10,6 % (avril) à +24,4 % (mars), autour de +16 % en médiane. L'écart n'est pas un effet de qualité mais de foncier — le prix de la maison intègre un terrain, dilué sur une surface bâtie plus grande.

Il n'y a pas de tendance annuelle exploitable. L'amplitude sur l'année est de 8,4 % pour l'appartement et 9,4 % pour la maison, du même ordre que la dispersion mensuelle. Le +5,2 % de janvier à décembre sur l'appartement tient à deux points extrêmes isolés (mars à 3 582, octobre à 3 315) et disparaît si on lisse. **Conclusion honnête : le marché héraultais du logement est stable en 2024, il ne monte pas.**

Le volume, lui, bouge nettement : 1 295 mutations en janvier contre 2 041 en décembre, soit +58 %.

Réserves. La date retenue est celle de la signature de l'acte, trois à quatre mois après l'accord sur le prix : la courbe est décalée d'un trimestre par rapport au marché réel, et la montée du volume en fin d'année est très probablement l'écho des compromis du printemps, pas une reprise de décembre. Une moyenne — et non une médiane — a été retenue ici pour rester lisible en série temporelle ; elle est plus sensible aux valeurs extrêmes que la médiane de Q1, ce que les bornes `100 < prix_m2 < 20 000` limitent sans l'annuler.

Question 3 — Dans un rayon de 5 km

Quels appartements se sont vendus à moins de 5 km du centre de Montpellier, et à quel prix au m² ?

```
{ $geoNear: {
  near: { type: "Point", coordinates: [3.8767, 43.6108] },
  distanceField: "distance_m",
  maxDistance: 5000,
  spherical: true,
  query: { nature_mutation: { $in: ["Vente", "Vente en l'état futur d'achèvement"] },
    type_local: "Appartement", surface_reelle_bati: { $gt: 9 },
    valeur_fonciere: { $gt: 1000 }, prix_m2: { $gt: 100, $lt: 20000 } } }
```

`$geoNear` doit être le **premier** stage du pipeline et exige l'index `2dsphere` : sans lui la requête échoue, elle ne se contente pas d'être lente.

Rendu sur une carte Leaflet, un cercle par mutation, le prix au m² en infobulle.

3 925 appartements vendus dans le rayon. Prix moyen au m² par tranche de distance :

Distance au centre	Prix moyen €/m ²	Ventes
0 – 1 km	3 747	768
1 – 2 km	3 305	1 184
2 – 3 km	3 308	1 007
3 – 4 km	3 149	653
4 – 5 km	3 731	313

Interprétation. Le résultat contredit l'intuition : **le prix n'est pas une fonction décroissante de la distance au centre**. Il chute de 12 % dès le premier kilomètre, reste sur un plateau entre 1 et 3 km, atteint son minimum entre 3 et 4 km — puis **remonte de 18 % entre 4 et 5 km**, jusqu'à retrouver le niveau de l'hypercentre.

Montpellier ne suit donc pas un modèle monocentrique mais un modèle **centre + pôles** : la couronne 4-5 km capte les programmes récents de l'est de la ville, dont les prix au neuf sont décorrélés de la distance à la Comédie. Un négociateur qui estimerait un bien au seul critère « distance au centre » se tromperait systématiquement sur cette couronne.

Réserves. La distance est mesurée à vol d'oiseau depuis un point arbitraire (la place de la Comédie) : elle ne dit rien de l'accessibilité réelle, et un rayon de 5 km ne recouvre pas un bassin de vie. La tranche 4-5 km ne repose que sur 313 ventes, contre 1 184 pour la tranche 1-2 km : la remontée est nette mais moins solidement établie que le plateau. Enfin, 235 mutations du jeu complet n'ont aucune coordonnée et sont invisibles sur la carte, sans que leur absence soit signalée à l'utilisateur.

Ce que la carte ajoute aux chiffres. Les mutations y sont colorées par tranche de prix (`captures/front.png`), et le motif obtenu corrige notre propre lecture : **le gradient n'est pas radial, il est directionnel**. Les ventes sous 2 500 €/m² forment un bloc compact à l'ouest et au sud-ouest — Les Cévennes, Croix d'Argent, Mosson — tandis que les ventes au-dessus de 4 500 €/m² se concentrent dans l'hypercentre et sur un axe est vers Port Marianne. À distance égale du centre, l'écart entre l'ouest et l'est dépasse celui entre le centre et la périphérie.

Le découpage par tranches de distance du tableau ci-dessus moyennait donc deux populations opposées dans chaque anneau, et le creux à 3-4 km n'est pas un creux : c'est le poids de l'ouest dans cet anneau. **Une agrégation par distance seule était le mauvais découpage pour cette question** — un `$geoWithin` sur des polygones de quartier répondrait mieux, et c'est ce que nous ferions avec une journée de plus.

Défaut corrigé en cours de route : la route renvoyait 500 résultats par défaut. Comme `$geoNear` trie par distance croissante, la carte s'arrêtait en réalité à 1,3 km et **n'affichait jamais la remontée des 4-5 km** — le résultat le plus intéressant de la question était masqué par une valeur par défaut.

Captures

```
explain("executionStats") sur db.mutations.find({ code_postal: "34000" }).
```

Avant, aucun index — extrait de `captures/explain_avant.json` :

```
"stage": "COLLSCAN",
"totalKeysExamined": 0,
"totalDocsExamined": 29565,
"nReturned": 1933,
"executionTimeMillis": 16
```

MongoDB lit les 29565 documents de la collection pour en retourner 1933 : il en examine **15,3 pour chacun qu'il rend**.

Après création de `{ code_postal: 1, date_mutation: -1 }` — `captures/explain_apres.json` :

```
"stage": "FETCH",
"inputStage": { "stage": "IXSCAN" },
"totalKeysExamined": 1933,
"totalDocsExamined": 1933,
"nReturned": 1933,
"executionTimeMillis": 4
```

Le ratio tombe à **1,0** : chaque document lu est un document rendu. Noter que le stage racine est `FETCH` et non `IXSCAN` — l'`IXSCAN` est son `inputStage`, et c'est la chaîne complète qui prouve que l'index sert.

Le front en fonctionnement : capture pleine page en annexe A, fichier `captures/front.png`.

v) Conclusion

Ce qui a été livré. Les six exigences du cahier des charges sont couvertes : 29 565 documents issus d'une source publique citée, deux collections liées par un `$lookup` argumenté, une API REST avec CRUD complet et trois routes d'agrégation, trois questions métier formulées avant leur pipeline, des index justifiés par `explain()`, et une authentification MongoDB avec un rôle applicatif restreint et les secrets hors du dépôt.

Ce que nous n'avons pas eu le temps de faire. Le validateur `$jsonSchema` sur `mutations`, la pagination du front, le `$graphLookup`, la mesure du coût en écriture de nos index, et surtout le découpage de Q3 par polygones de quartier (`$geoWithin`) plutôt que par anneaux de distance — c'est la limite méthodologique que la carte nous a révélée trop tard.

Passage à l'échelle. À 10 millions de documents — l'équivalent de DVF sur toute la France et dix ans — la collection dépasserait ce qu'une instance unique sert confortablement. La shard key naturelle est `{ code_commune: 1, date_mutation: 1 }` : cardinalité élevée (35 000 communes), pas de hot spot en écriture puisque les mutations arrivent réparties sur tout le territoire, et surtout **elle correspond à nos filtres dominants**. Q1 et Q3 resteraient *targeted* — elles filtrent par commune ou par zone, donc par un préfixe de la clé. Q2, qui agrège sur toute la France par mois,

deviendrait un *broadcast* : c'est le prix à payer, et il est acceptable pour une requête de tableau de bord exécutée une fois par jour. Une shard key sur la seule `date_mutation` serait le pire choix possible : monotone croissante, elle enverrait toutes les écritures sur un unique shard.

Limites du travail. Nos données ne permettent pas de conclure sur le foncier nu (35 % des mutations écartées), ni sur les mutations à titre gratuit (absentes de DVF), ni sur le prix réel au moment de l'accord (décalage de trois à quatre mois avec la date d'acte). Elles ne disent rien de l'état des biens : un T3 rénové et un T3 à refaire ont le même prix au m² dans ce jeu.

Ce que nous ferions différemment. Écrire la version naïve et la version correcte de la requête principale **dès la première heure**, et non en fin de journée : c'est ce qui a révélé le facteur 3,60, et donc la nécessité du regroupement. Nous avons découvert le problème après avoir écrit trois pipelines qu'il a fallu reprendre.

Annexes

Hors pagination.

Schéma détaillé des collections : § iii · liste complète des routes : `docs/API.md` · commandes d'import : `db/import.sh` · répartition du travail : `README.md`.

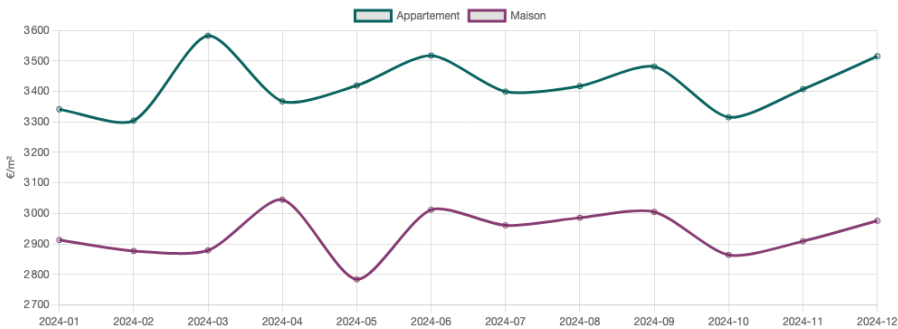
A. Le front en fonctionnement

Les trois vues sont alimentées par les trois routes `/agg/*` de l'API, sur la base réelle : le tableau de Q1, le graphique de Q2 et la carte de Q3, colorée par tranche de prix au m².

Q1 · Les 10 communes les plus chères (≥ 30 ventes)

COMMUNE	PRIX MÉDIAN €/M²	VENTES
La Grande-Motte	5 400	664
Palavas-les-Flots	5 135	234
Vic-la-Gardiole	5 128	65
Lattes	4 581	224
Mauguio	4 542	325
Pérols	4 375	120
Marseillan	4 150	301
Balaruc-les-Bains	4 126	225
Clapiers	3 948	44
Castelnau-le-Lez	3 947	355

Q2 · Évolution mensuelle du prix moyen au m²



Q3 · Appartements vendus à moins de 5 km du centre de Montpellier

3 925 appartements vendus

